

Introduction to Matplotlib

What is Matplotlib?

Matplotlib is a powerful Python library used for data visualization. It allows users to create a wide variety of static, animated, and interactive plots in Python. It is particularly useful in data analysis and machine learning projects. Matplotlib was created by John D. Hunter.

Matplotlib is open source and we can use it freely.

Matplotlib is mostly written in python, a few segments are written in C, Objective-C and Javascript for Platform compatibility.

Installing Matplotlib

Before using Matplotlib, ensure you have it installed. You can install it using pip:

```
pip install matplotlib
```

Importing Matplotlib

To use Matplotlib, import it in your Python script. The most commonly used module in Matplotlib is `pyplot`, typically imported as `plt`:

```
import matplotlib.pyplot as plt
```

Pyplot

Most of the Matplotlib utilities lies under the `pyplot` submodule, and are usually imported under the `plt` alias:

```
import matplotlib.pyplot as plt
```

Basic Plotting with Matplotlib

Creating a Simple Line Plot

A line plot is the most basic type of plot. Here's an example:

```
import matplotlib.pyplot as plt
# Sample data
x = [1, 2, 3, 4, 5]
y = [10, 20, 25, 30, 40]
# Creating the plot
plt.plot(x, y)
# Adding labels and title
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Simple Line Plot")
# Display the plot
plt.show()
```

Example:

Draw a line in a diagram from position (0,0) to position (6,250):

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([0, 6])
ypoints = np.array([0, 250])
plt.plot(xpoints, ypoints)
plt.show()
```

1. Plotting x and y Points

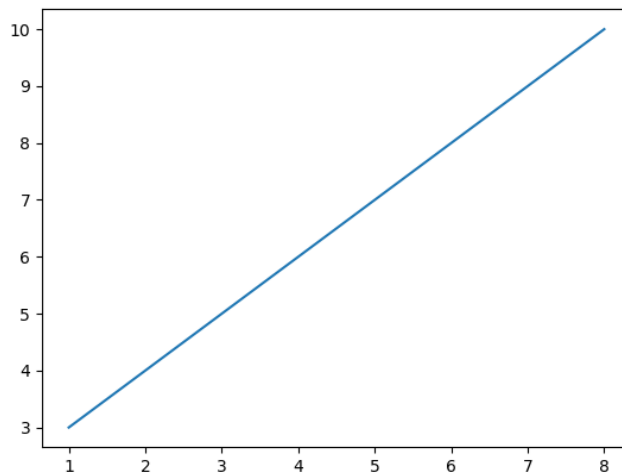
To draw points in a graph, we use the `plot()` function.

- The **first parameter** represents the x-axis values (horizontal positions).
- The **second parameter** represents the y-axis values (vertical positions).
- By default, `plot()` connects the points with a straight line.

Example: Drawing a Line Between Two Points

Suppose we want to draw a line from point (1,3) to (8,10)

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 8]) # X-axis values
ypoints = np.array([3, 10]) # Y-axis values
plt.plot(xpoints, ypoints) # Plot the points with a line
plt.show() # Display the graph
```



Output:

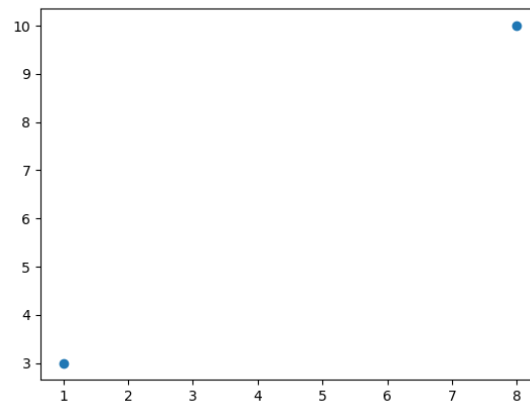
A straight line connecting (1,3) and (8,10).

2. Plotting Only Points (Without a Line)

If you want to show only the points without connecting them, use 'o' as a parameter in `plot()`. The 'o' means "rings" (circle markers).

Example: Plotting Two Points Without a Line

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 8])
ypoints = np.array([3, 10])
plt.plot(xpoints, ypoints, 'o') # Only plot the points
plt.show()
```



Output:

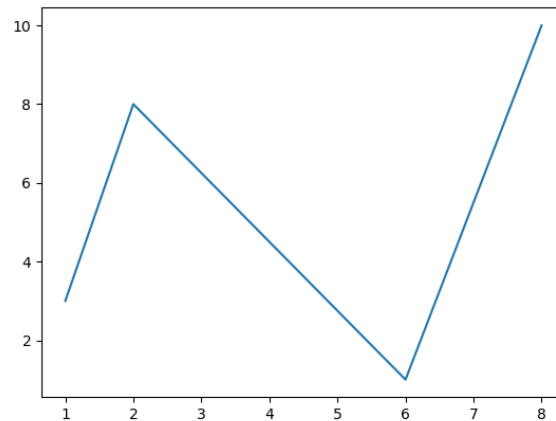
Two separate circles at (1,3) and (8,10).

3. Plotting Multiple Points

We can add more points and connect them to form a path.

Example: Drawing a Line Through Multiple Points

```
import matplotlib.pyplot as plt
import numpy as np
xpoints = np.array([1, 2, 6, 8])
ypoints = np.array([3, 8, 1, 10])
plt.plot(xpoints, ypoints) # Connect all points with a line
plt.show()
```



Output:

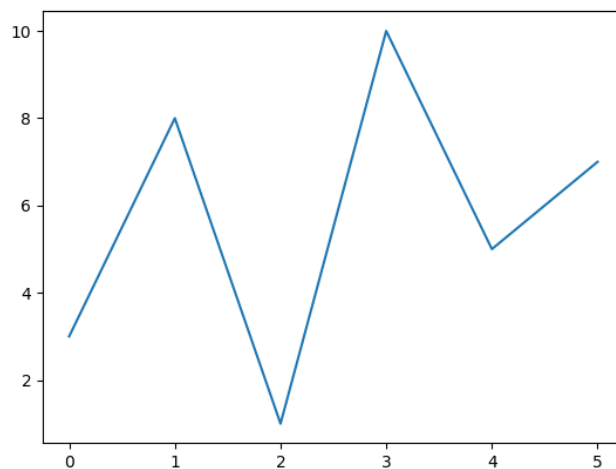
A zig-zag line passing through $(1,3) \rightarrow (2,8) \rightarrow (6,1) \rightarrow (8,10)$.

4. Default X-Points

If we don't provide x-values, Python automatically assigns default x-values starting from **0, 1, 2, 3, ...**

Example: Plotting Without X-Points

```
import matplotlib.pyplot as plt
import numpy as np
ypoints = np.array([3, 8, 1, 10, 5, 7])
plt.plot(ypoints) # X-values are automatically set to [0, 1, 2, 3, 4, 5]
plt.show()
```



Output:

Points are plotted at $(0,3)$, $(1,8)$, $(2,1)$, $(3,10)$, $(4,5)$, and $(5,7)$.

Summary

- `plot(x, y)` → Plots a line through given points.
- `plot(x, y, 'o')` → Plots only points (no line).
- Multiple points can be connected in sequence.
- If x-values are not given, they start from 0 automatically.

Customizing the Plot/Matplotlib Markers

You can customize the appearance of your plots using different parameters:

```
plt.plot(x, y, marker='o', linestyle='--', color='r', linewidth=2, markersize=8)
```

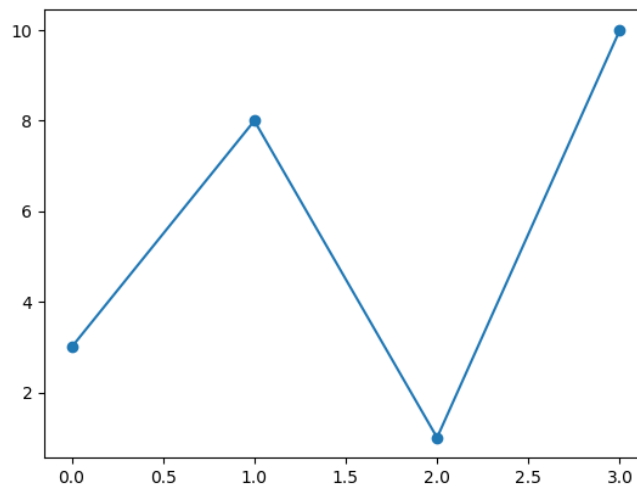
- `marker='o'` adds circular markers.
- `linestyle='--'` creates a dashed line.
- `color='r'` makes the line red.
- `linewidth=2` controls the thickness.
- `markersize=8` adjusts the size of markers.

Different Marker Styles

Marker	Symbol
'o'	Circle
'*'	Star
'.'	Small dot
','	Pixel dot
'x'	Cross (X)
's'	Square
'D'	Diamond
'^'	Triangle Up
'v'	Triangle Down

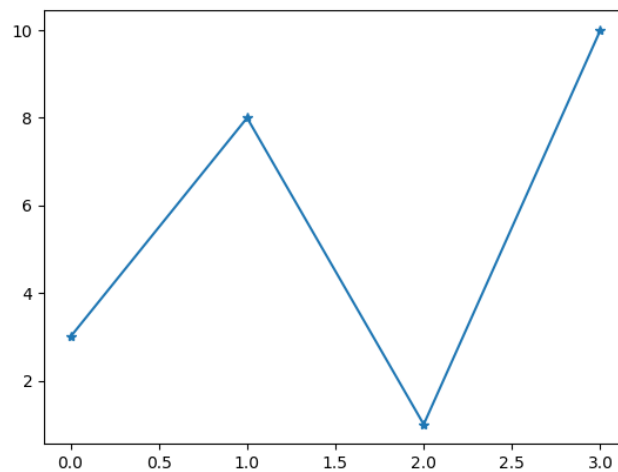
Add Circles at Each Point

```
ypoints = np.array([3, 8, 1, 10])  
plt.plot(ypoints, marker='o') # 'o' adds circle markers  
plt.show()
```



Add Circles at Each Point

```
plt.plot(ypoints, marker='*') # Use '*' for star markers  
plt.show()
```



Using Format Strings (Shortcut for Markers, Lines, and Colors)

You can **combine** marker, line, and color in one shortcut string.

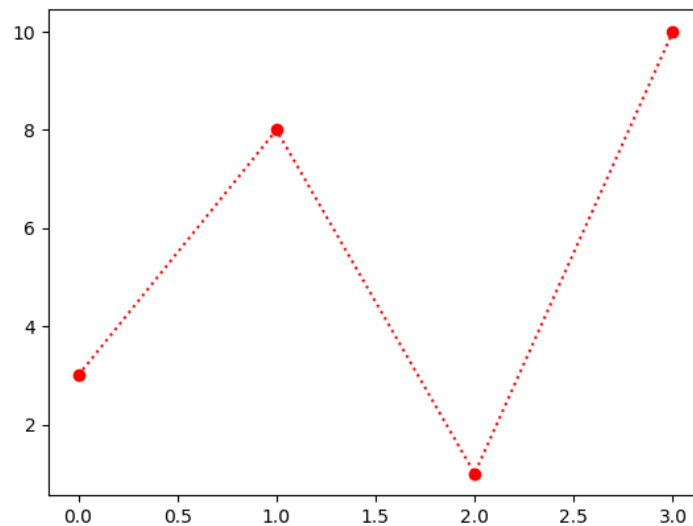
Syntax:

marker | line style | color

Element	Syntax	Example
Marker	'o', '*', 's'	'o' for circle
Line	'-', '--', ':', '-.'	'--' for dashed line
Color	'r', 'g', 'b', 'c', 'm', 'y', 'k', 'w'	'r' for red

Example: Red Dotted Line with Circle Markers

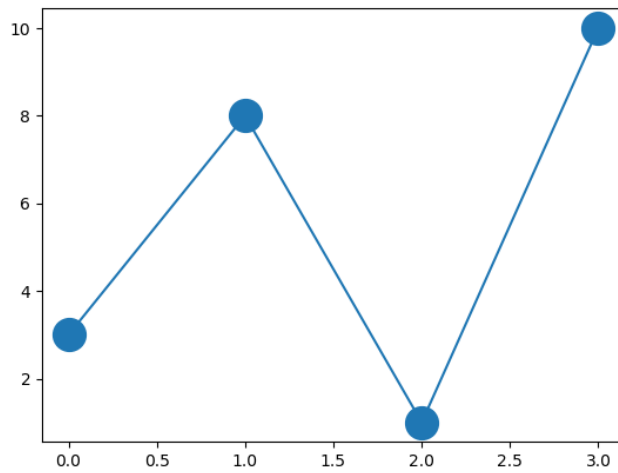
```
plt.plot(ypoints, 'o:r') # Circle markers, dotted line, red color
plt.show()
```



Changing Marker Size

The `markersize` or `ms` argument adjusts **how big** the markers appear.

```
plt.plot(ypoints, marker='o', ms=20) # Bigger circles (20 size)
plt.show()
```

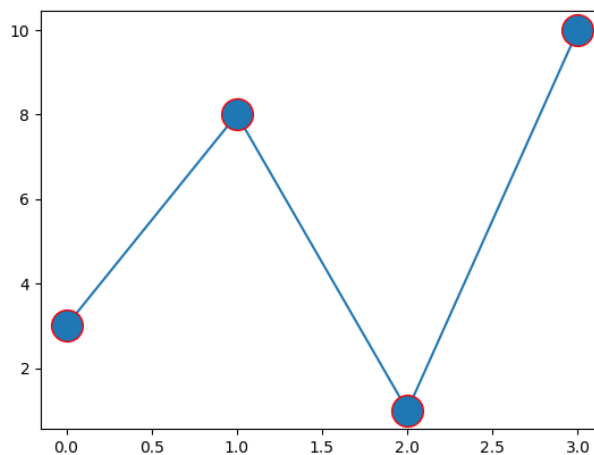
Changing Marker Colors

You can **change marker edge color and face color**.

Argument	What it Does
mec	Marker Edge Color
mfc	Marker Face Color

Example: Red Edge with Blue Fill

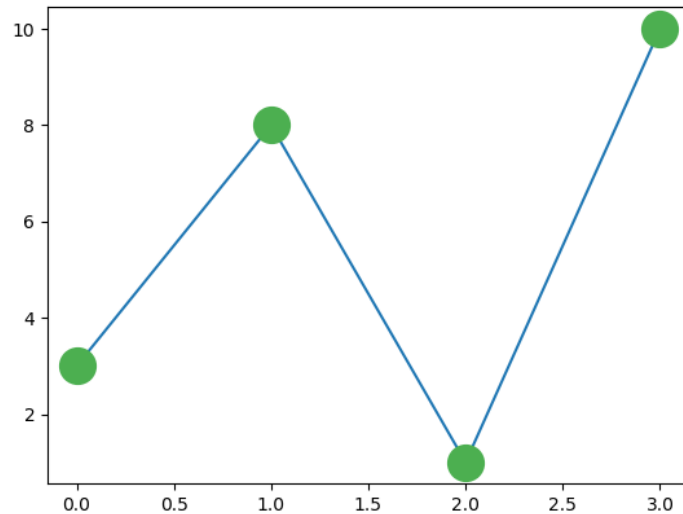
```
plt.plot(ypoints, marker='o', ms=20, mec='r', mfc='b') # Red border, blue inside  
plt.show()
```



Using Custom Hex Color Codes

Instead of color names, you can use **Hex codes**:

```
plt.plot(ypoints, marker='o', ms=20, mec='#4CAF50', mfc='#4CAF50') # Green color  
plt.show()
```

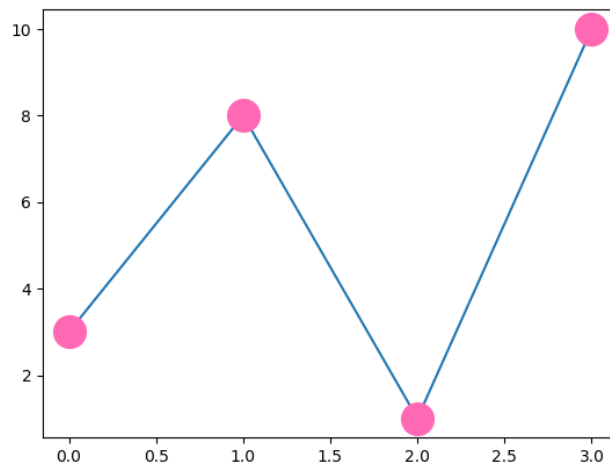


Using Named Colors

Matplotlib supports **140+ color names** like 'hotpink', 'skyblue', 'orange'.

Example: Hot Pink Markers

```
plt.plot(ypoints, marker='o', ms=20, mec='hotpink', mfc='hotpink')  
plt.show()
```



Feature Example

Basic Line Plot	<code>plt.plot(x, y)</code>
Only Points (No Line)	<code>plt.plot(x, y, 'o')</code>
Multiple Points	<code>plt.plot([1,2,6,8], [3,8,1,10])</code>
Default X-Values	<code>plt.plot([3,8,1,10])</code>
Different Markers	<code>plt.plot(x, y, marker='*')</code>
Marker + Line + Color	<code>plt.plot(x, y, 'o--r')</code>
Marker Size	<code>plt.plot(x, y, marker='o', ms=20)</code>
Marker Edge & Face Color	<code>plt.plot(x, y, marker='o', mec='r', mfc='b')</code>
Hex Colors	<code>plt.plot(x, y, marker='o', mfc='#4CAF50')</code>
Named Colors	<code>plt.plot(x, y, marker='o', mfc='hotpink')</code>

Line Styles (Connecting the Points)

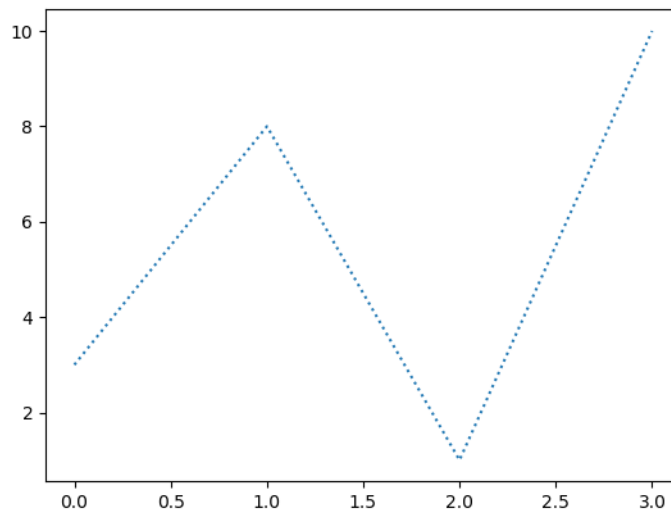
By default, Matplotlib connects data points with a straight **solid line**. You can change the style using `linestyle` or `ls`.

Different Line Styles

Line Style Code	Description
'-'	Solid (default)
'--'	Dashed
'.'	Dotted
'-.'	Dash-dot

Example: Use a Dotted Line

```
plt.plot(ypoints, linestyle='dotted')  
plt.show()  
plt.plot(ypoints, ls=':')  
plt.show()
```

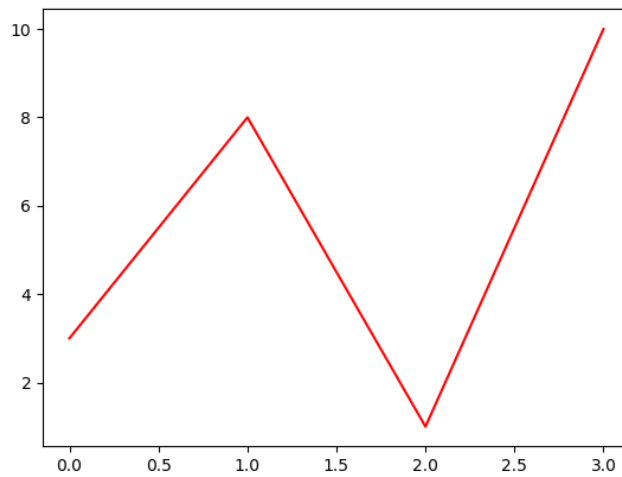


Line Colors

You can set the line color using color or c.

Example: Red Line

```
plt.plot(ypoints, color='red') # or c='r'  
plt.show()
```

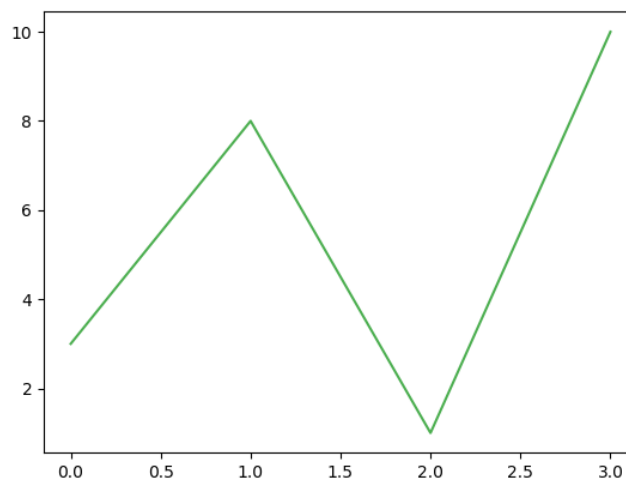


Other Color Options:

- 'r' = Red
- 'g' = Green
- 'b' = Blue
- 'c' = Cyan
- 'm' = Magenta
- 'y' = Yellow
- 'k' = Black
- 'w' = White

Example: Hex Color Code

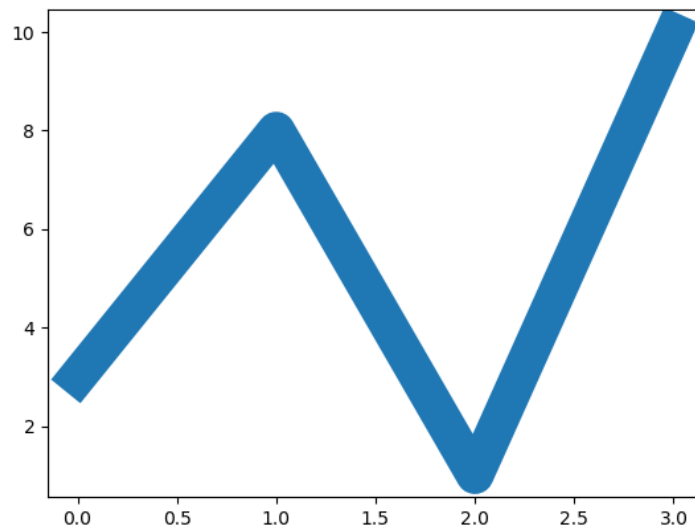
```
plt.plot(ypoints, c='#4CAF50') # Green shade  
plt.show()
```



Line Width

The width of the line can be changed using `linewidth` or `lw`.

```
plt.plot(yvalues, linewidth=3) # Thick line
plt.show()
```

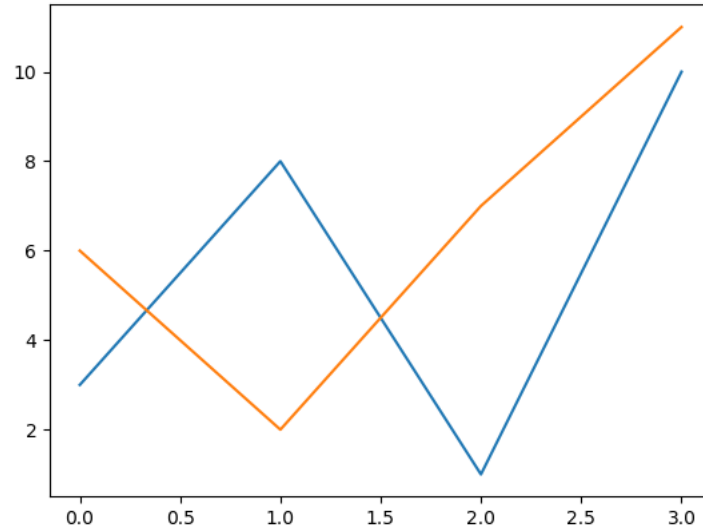


Multiple Lines on the Same Graph

You can plot multiple lines by calling `plt.plot()` multiple times or by providing multiple x and y values.

Example: Two Lines

```
y1 = np.array([3, 8, 1, 10])
y2 = np.array([6, 2, 7, 11])
plt.plot(y1, label="Line 1") # First line
plt.plot(y2, label="Line 2") # Second line
plt.legend() # Show labels
plt.show()
```

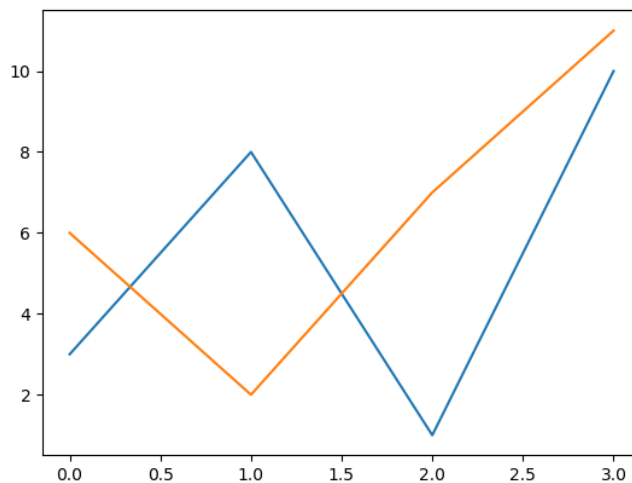


Explanation:

- Two lines are plotted on the same graph.
- `plt.legend()` adds labels to identify each line.

Example: Specify x and y Values for Two Lines

```
x1 = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])
x2 = np.array([0, 1, 2, 3])
y2 = np.array([6, 2, 7, 11])
plt.plot(x1, y1, x2, y2)
plt.show()
```



Types of Plots in Matplotlib

1. Bar Chart

A bar chart represents categorical data.

```
categories = ['A', 'B', 'C', 'D']
values = [5, 7, 3, 8]
plt.bar(categories, values, color='blue')
plt.xlabel("Categories")
plt.ylabel("Values")
plt.title("Bar Chart Example")
plt.show()
```

2. Scatter Plot

A scatter plot is used to represent relationships between two numerical variables.

```
import numpy as np
x = np.random.rand(50)
y = np.random.rand(50)
plt.scatter(x, y, color='green')
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Scatter Plot Example")
plt.show()
```

3. Histogram

Histograms show the distribution of data.

```
data = np.random.randn(1000)
plt.hist(data, bins=30, color='purple', alpha=0.7)
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.title("Histogram Example")
plt.show()
```

4. Pie Chart

A pie chart represents proportions.

```
labels = ['Apples', 'Bananas', 'Cherries', 'Dates']
sizes = [30, 20, 40, 10]
plt.pie(sizes, labels=labels, autopct='%1.1f%%', colors=['red', 'yellow',
'green', 'brown'])
plt.title("Pie Chart Example")
plt.show()
```


Subplots in Matplotlib

Sometimes, we need multiple plots in the same figure.

```
fig, axs = plt.subplots(2, 2)
axs[0, 0].plot(x, y, 'r') # Red line plot
axs[0, 1].bar(categories, values) # Bar chart
axs[1, 0].scatter(x, y, color='blue') # Scatter plot
axs[1, 1].hist(data, bins=20, color='green') # Histogram
plt.tight_layout()
plt.show()
```

In Matplotlib, the `subplot()` function lets you display **multiple plots in one figure** (a single window). This is useful when you want to compare different graphs side by side or on top of each other.

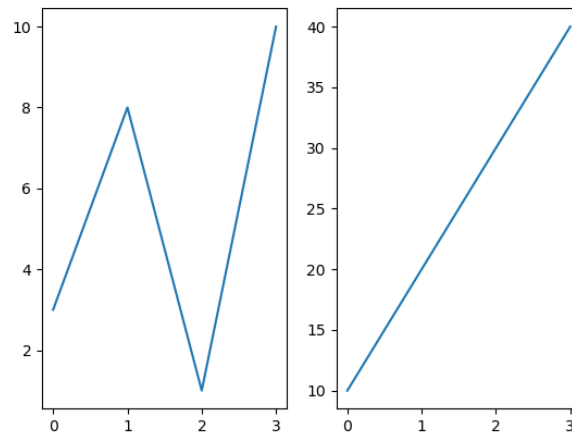
How subplot() Works

The `subplot(rows, columns, index)` function takes **three numbers**:

1. **Rows** – How many rows of plots do you want?
2. **Columns** – How many columns of plots do you want?
3. **Index** – Which plot are you currently working on?

Example 1: Two Plots Side by Side

```
import matplotlib.pyplot as plt
import numpy as np
# Data for the first plot
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(1, 2, 1) # 1 row, 2 columns, plot 1
plt.plot(x, y)
# Data for the second plot
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
plt.subplot(1, 2, 2) # 1 row, 2 columns, plot 2
plt.plot(x, y)
plt.show()
```



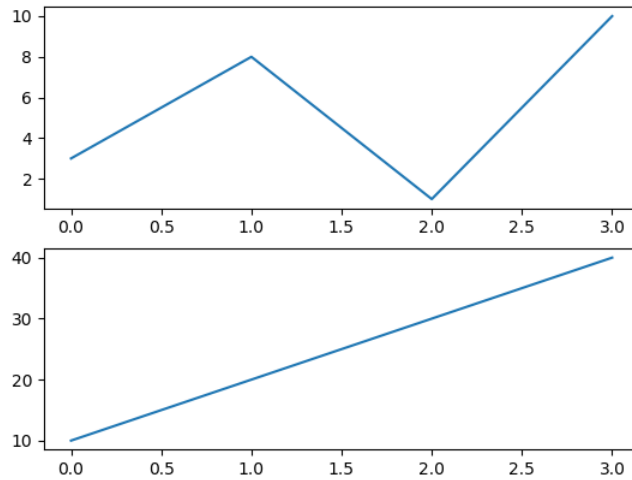
How It Works:

- The figure has **1 row and 2 columns** (so the plots are **side by side**).
- `subplot(1, 2, 1)` → First plot (on the left).
- `subplot(1, 2, 2)` → Second plot (on the right).

Example 2: Two Plots on Top of Each Other

```
import matplotlib.pyplot as plt
import numpy as np
```

```
# First plot
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(2, 1, 1) # 2 rows, 1 column, first plot
plt.plot(x, y)
# Second plot
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
plt.subplot(2, 1, 2) # 2 rows, 1 column, second plot
plt.plot(x, y)
plt.show()
```



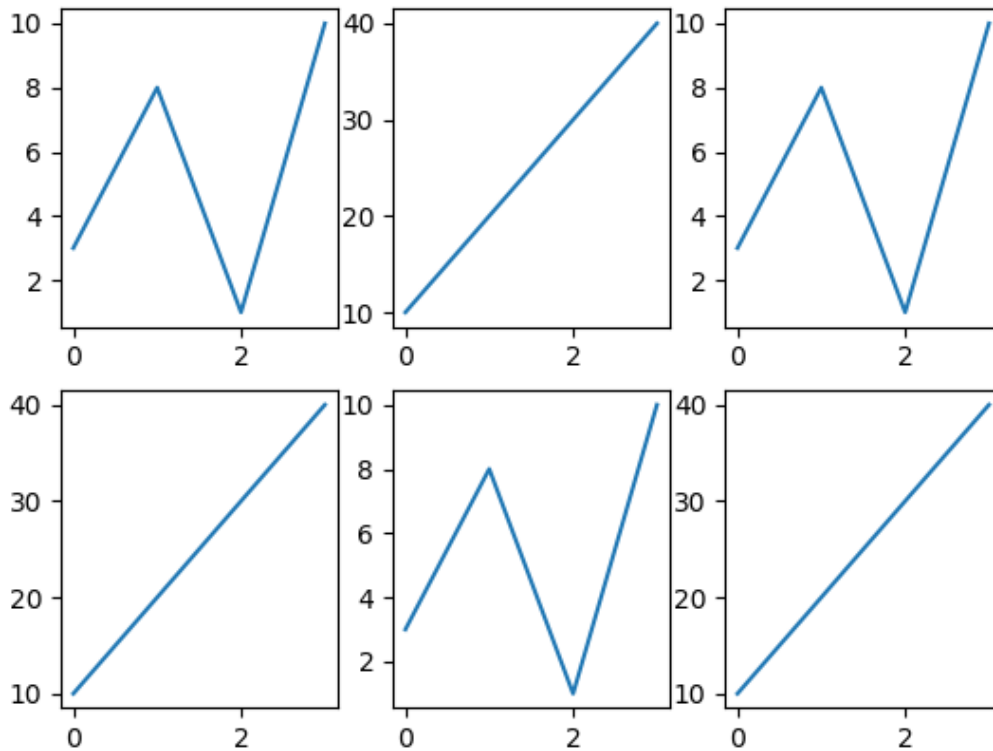
How It Works:

- The figure has **2 rows and 1 column** (so the plots are **stacked vertically**).
- `subplot(2, 1, 1)` → First plot (on top).
- `subplot(2, 1, 2)` → Second plot (below the first one).

Example 3: Multiple Plots in a Grid Layout

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array([0, 1, 2, 3])
y1 = np.array([3, 8, 1, 10])
y2 = np.array([10, 20, 30, 40])
plt.subplot(2, 3, 1) # 2 rows, 3 columns, plot 1
plt.plot(x, y1)
plt.subplot(2, 3, 2) # 2 rows, 3 columns, plot 2
plt.plot(x, y2)
plt.subplot(2, 3, 3) # 2 rows, 3 columns, plot 3
plt.plot(x, y1)
plt.subplot(2, 3, 4) # 2 rows, 3 columns, plot 4
plt.plot(x, y2)
plt.subplot(2, 3, 5) # 2 rows, 3 columns, plot 5
plt.plot(x, y1)
plt.subplot(2, 3, 6) # 2 rows, 3 columns, plot 6
plt.plot(x, y2)
```

plt.show()



How It Works:

- The figure has **2 rows and 3 columns** (so the plots form a **grid**).
- Each subplot(2, 3, index) places a plot in a **specific location**.

Adding Titles to Plots in Matplotlib

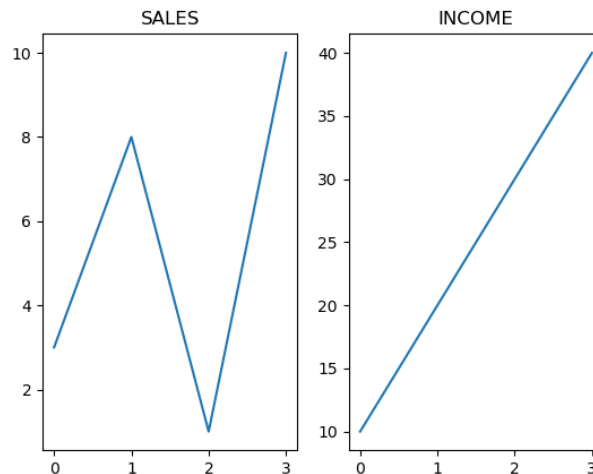
Adding a Title to Each Plot:

You can use `plt.title("Your Title")` to add a **title** to individual plots.

Example: Two Plots with Titles

```
import matplotlib.pyplot as plt
import numpy as np
# First plot (Sales)
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(1, 2, 1) # 1 row, 2 columns, first plot
plt.plot(x, y)
```

```
plt.title("SALES") # Title for this plot
# Second plot (Income)
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
plt.subplot(1, 2, 2) # 1 row, 2 columns, second plot
plt.plot(x, y)
plt.title("INCOME") # Title for this plot
plt.show()
```



How It Works:

- ✓ Each plot has its own title using `plt.title()`.
- ✓ The **first plot** is titled "SALES".
- ✓ The **second plot** is titled "INCOME".

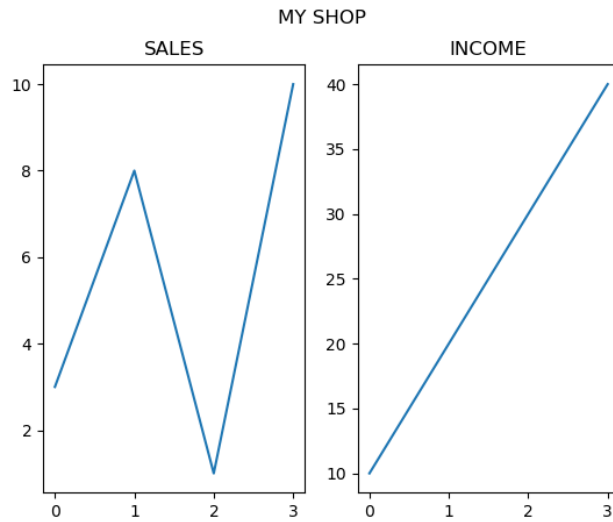
Adding a Main Title for All Plots

You can use `plt.suptitle("Your Main Title")` to add a **title for the entire figure**.

Example: Main Title for the Figure

```
import matplotlib.pyplot as plt
import numpy as np
# First plot (Sales)
x = np.array([0, 1, 2, 3])
y = np.array([3, 8, 1, 10])
plt.subplot(1, 2, 1)
plt.plot(x, y)
plt.title("SALES")
# Second plot (Income)
x = np.array([0, 1, 2, 3])
y = np.array([10, 20, 30, 40])
plt.subplot(1, 2, 2)
plt.plot(x, y)
plt.title("INCOME")
```

```
plt.suptitle("MY SHOP") # Main title for the figure  
plt.show()
```



How It Works:

- ✓ `plt.suptitle("MY SHOP")` adds a **title for the entire figure**.
- ✓ The individual plots still have their own titles.
- ✓ The result looks more organized and professional.

Adding Legends, Grids, and Annotations

1.Adding a Legend

```
plt.plot(x, y, label='Line 1')  
plt.plot(x, [i+5 for i in y], label='Line 2')  
plt.legend()  
plt.show()
```

2.Adding a Grid

```
plt.plot(x, y)  
plt.grid(True)  
plt.show()
```

3.Adding Annotations

```
plt.plot(x, y)  
plt.annotate('Peak', xy=(4, 30), xytext=(3, 35),  
            arrowprops=dict(facecolor='black'))  
plt.show()
```

Matplotlib Labels and Titles

When making graphs with Matplotlib, you can **add labels** to the X and Y axes and a **title** to the chart to make it clear and informative.

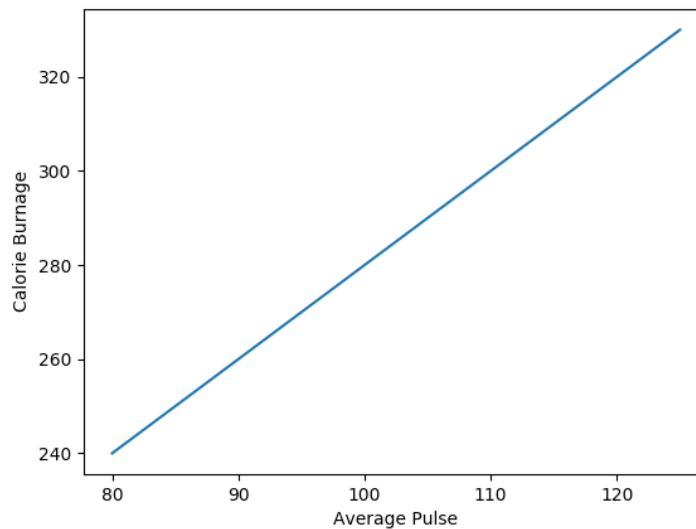
Adding Labels to X and Y Axis

You can use:

- `plt.xlabel("Your X Label")` → To label the **X-axis**
- `plt.ylabel("Your Y Label")` → To label the **Y-axis**

Example: Adding Labels

```
import numpy as np
import matplotlib.pyplot as plt
# Data points
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
plt.plot(x, y)
# Adding labels
plt.xlabel("Average Pulse") # X-axis label
plt.ylabel("Calorie Burnage") # Y-axis label
plt.show()
```



How It Works:

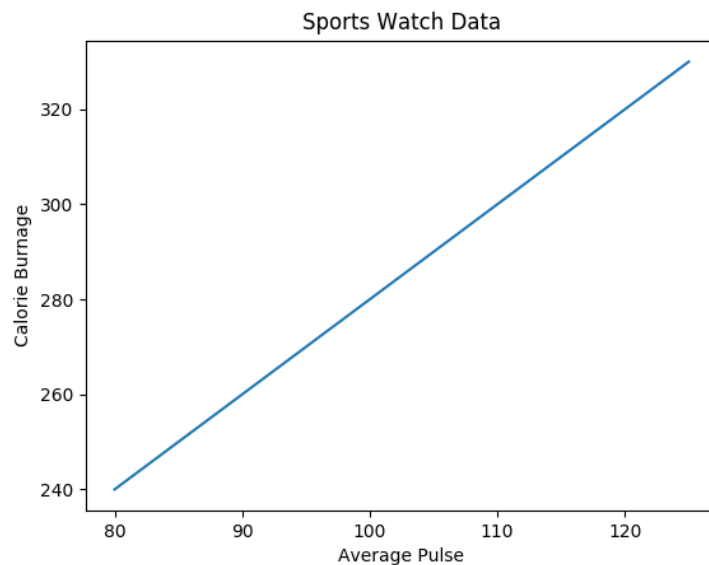
- ✓ The X-axis is labeled as "**Average Pulse**".
- ✓ The Y-axis is labeled as "**Calorie Burnage**".
- ✓ Helps understand what the graph represents.

Adding a Title to the Plot

You can use `plt.title("Your Title")` to add a **title** to your graph.

Example: Title + Labels

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
plt.plot(x, y)
plt.title("Sports Watch Data") # Adding a title
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")
```



```
plt.show()
```

How It Works:

- ✓ The title **"Sports Watch Data"** describes the graph.
- ✓ Labels **explain what the X and Y values represent.**

Changing Font Style of Title & Labels

You can use the `fontdict` parameter to customize **font size, color, and style.**

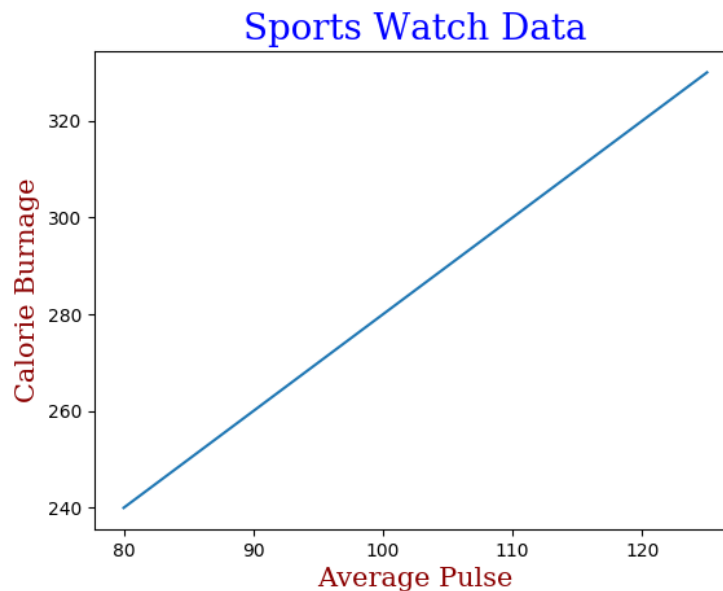
Example: Custom Fonts

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
# Font settings
title_font = {'family':'serif', 'color':'blue', 'size':20}
```

```
label_font = {'family':'serif', 'color':'darkred', 'size':15}

plt.title("Sports Watch Data", fontdict=title_font) # Styled title
plt.xlabel("Average Pulse", fontdict=label_font) # Styled X-axis label
plt.ylabel("Calorie Burnage", fontdict=label_font) # Styled Y-axis label

plt.plot(x, y)
plt.show()
```



How It Works:

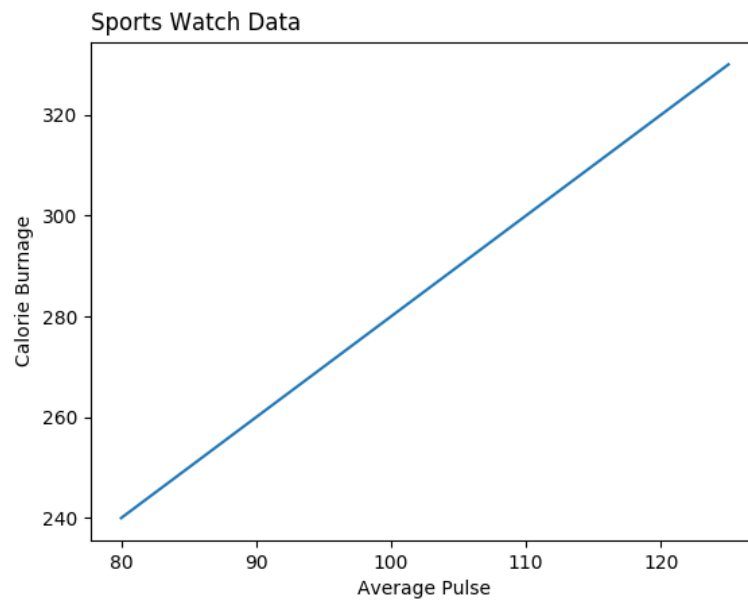
- ✅ The **title is blue and bigger (size: 20)**.
- ✅ The **axis labels are dark red (size: 15)**.
- ✅ The graph looks more attractive and readable! 🧠

Changing Title Position

You can **move the title to the left, right, or center** using loc in plt.title().

Example: Move Title to Left

```
import numpy as np
import matplotlib.pyplot as plt
x = np.array([80, 85, 90, 95, 100, 105, 110, 115, 120, 125])
y = np.array([240, 250, 260, 270, 280, 290, 300, 310, 320, 330])
plt.title("Sports Watch Data", loc='left') # Title aligned to the left
plt.xlabel("Average Pulse")
plt.ylabel("Calorie Burnage")
plt.plot(x, y)
plt.show()
```



How It Works:

- ✓ The **title is positioned to the left**.
- ✓ You can use 'left', 'center', or 'right' for positioning.

Matplotlib Bar Charts

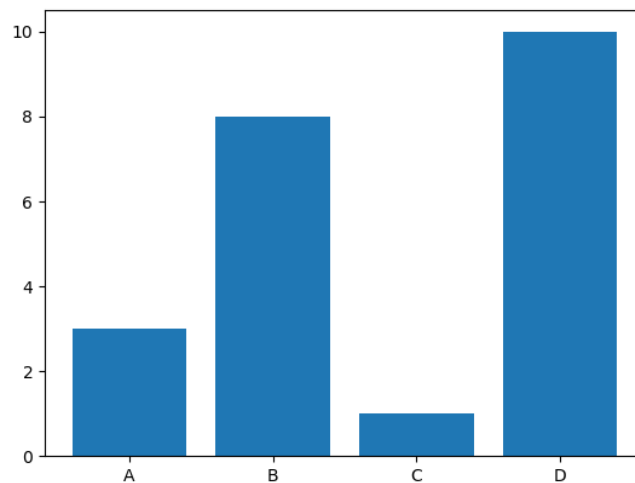
Bar charts are used to visualize data by representing categories with rectangular bars. In Python, we use **Matplotlib** to create bar charts easily.

1. Creating a Simple Bar Chart

To draw a **vertical** bar chart, use the `bar()` function in Matplotlib. Each bar represents a category with a specific value.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
# Categories (X-axis)
x = np.array(["A", "B", "C", "D"])
# Values (Y-axis)
y = np.array([3, 8, 1, 10])
# Create the bar chart
plt.bar(x, y)
# Show the chart
plt.show()
```



Explanation:

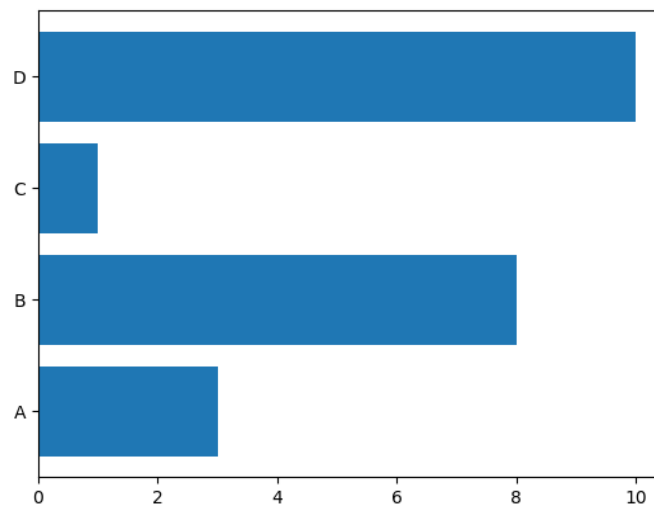
- "A", "B", "C", "D" are category names.
- The values [3, 8, 1, 10] determine the height of each bar.

2. Creating a Horizontal Bar Chart

If you want **horizontal** bars instead of vertical ones, use `barh()` instead of `bar()`.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
plt.barh(x, y) # 'barh' for horizontal bars
plt.show()
```



Key Difference:

- `bar()` → Vertical bars
- `barh()` → Horizontal bars

3. Changing Bar Colors

You can change the color of the bars using the `color` parameter.

Example:

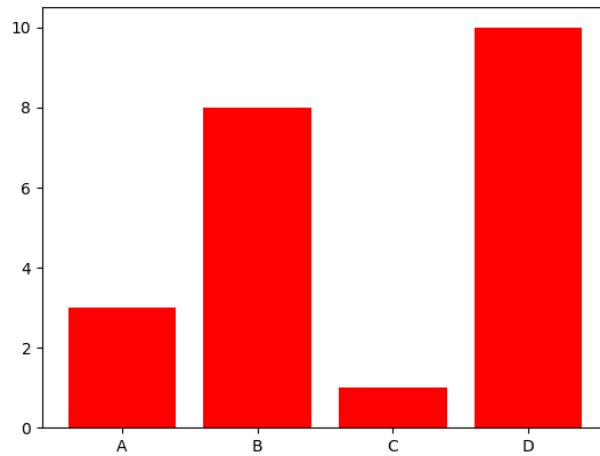
```
plt.bar(x, y, color="red") # Makes bars red
```

Other ways to define colors:

1. **Color Names:** "blue", "green", "hotpink", etc.
2. **Hex Color Codes:** "#4CAF50" (green color)

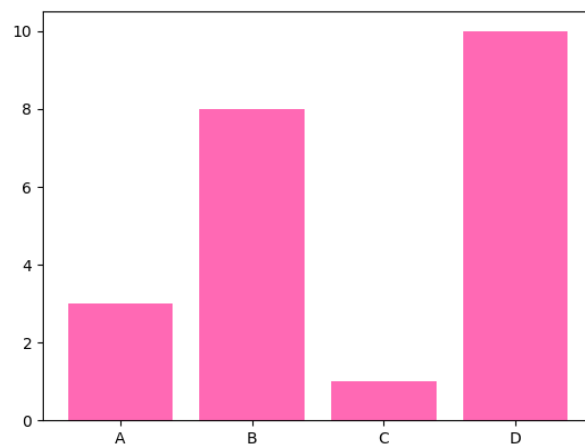
Example with a custom color:

```
plt.bar(x, y, color="#4CAF50") # Custom green color
```



Example: Draw 4 "hot pink" bars:

```
import matplotlib.pyplot as plt
import numpy as np
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
plt.bar(x, y, color = "hotpink")
plt.show()
```



4. Adjusting Bar Size

- For **vertical bars**, use `width` to set how wide the bars are.
- For **horizontal bars**, use `height`.

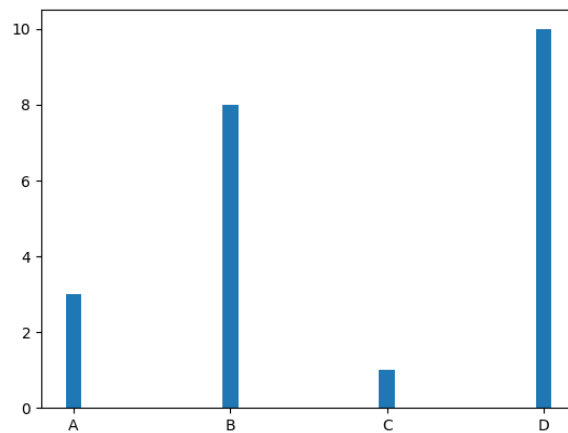
Example: Very thin vertical bars

The `bar()` takes the keyword argument `width` to set the width of the bars:

Example

Draw 4 very thin bars:

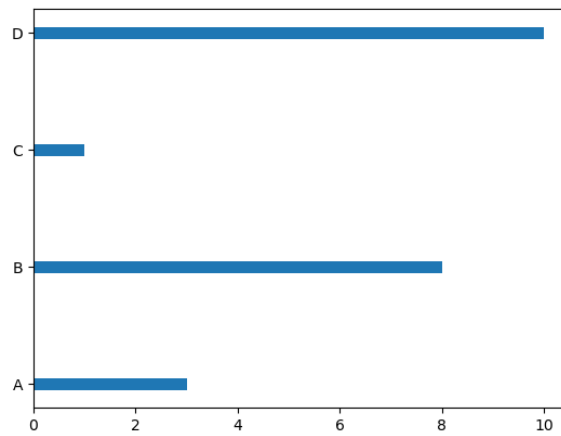
```
import matplotlib.pyplot as plt
import numpy as np
x = np.array(["A", "B", "C", "D"])
y = np.array([3, 8, 1, 10])
plt.bar(x, y, width = 0.1)
plt.show()
```



`plt.bar(x, y, width=0.1) # Very thin bars`

Example: Very thin horizontal bars

`plt.barh(x, y, height=0.1) # Very thin bars`



Default Width/Height: 0.8
Smaller values = thinner bars
Larger values = thicker bars

Matplotlib Histograms

A **histogram** is a type of graph that shows how often something happens. It groups numbers into **ranges (bins)** and then counts how many numbers fall into each range.

For example, if we ask **250 people** about their **height**, we might get different values. A histogram will show us how many people are within specific height ranges, like:

- 140 cm - 145 cm → 2 people
- 145 cm - 150 cm → 5 people
- 150 cm - 155 cm → 15 people
- ... and so on.

How to Create a Histogram in Python

In Python, we can use **Matplotlib** to create a histogram. The function we use is `hist()`.

Step 1: Generate Sample Data

Since we don't have real height data, we use **NumPy** to generate **random numbers** that are similar to real-world height values.

```
import numpy as np # Import NumPy library
# Generate 250 random height values with an average (mean) of 170 cm
# and a standard deviation of 10 cm
x = np.random.normal(170, 10, 250)
print(x) # This will print random height values
```

This gives us **random height values** where most people are around **170 cm** but some may be shorter or taller.

Step 2: Create the Histogram

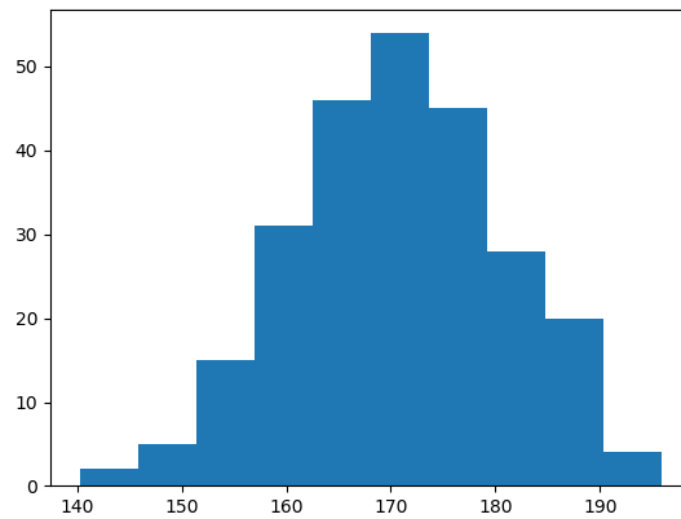
Now, we use **Matplotlib** to plot the histogram:

```
import matplotlib.pyplot as plt # Import Matplotlib library
# Create histogram
plt.hist(x) # Plot the histogram using our height data
# Show the histogram
plt.show()
```

Understanding the Output

- The x-axis represents **height ranges** (e.g., 140-145 cm, 145-150 cm, etc.).
- The y-axis represents **how many people** fall into each height range.

- The taller bars show where most people's heights are.



Matplotlib Pie Charts

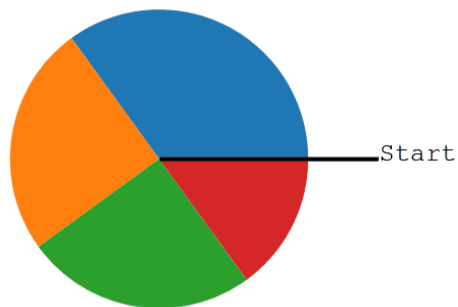
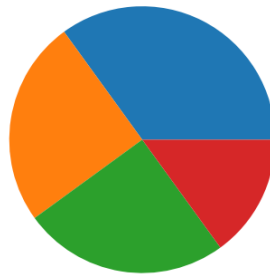
A **pie chart** is a circular graph that represents data as slices (wedges). Each slice's size is proportional to its value compared to the whole. Let's explore how to create pie charts step by step using **Matplotlib** in Python.

1. Creating a Basic Pie Chart

You can make a simple pie chart using the `plt.pie()` function.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
# Data values (percentages)
y = np.array([35, 25, 25, 15])
# Create a pie chart
plt.pie(y)
# Show the chart
plt.show()
```



Explanation:

- `plt.pie(y)` draws the pie chart with given values.
- `plt.show()` displays the chart.

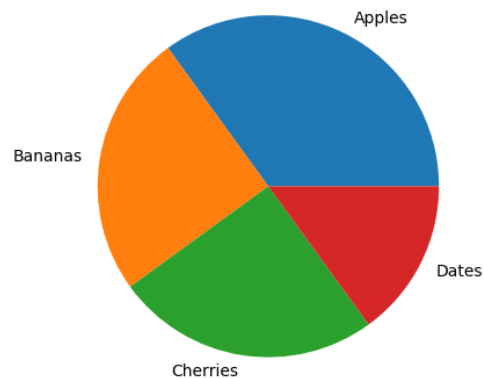
Each number in the array represents a **portion of the whole**. The chart automatically calculates how much space each slice should take.

2. Adding Labels

Labels help identify what each slice represents.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]
plt.pie(y, labels=labels)
plt.show()
```



Explanation:

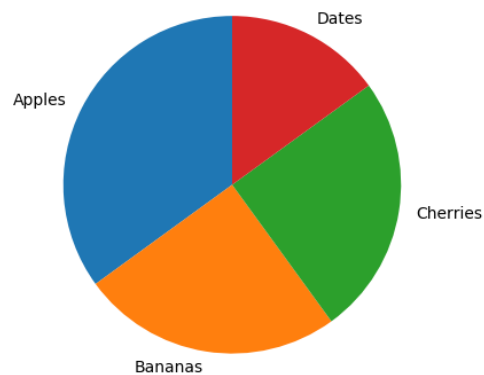
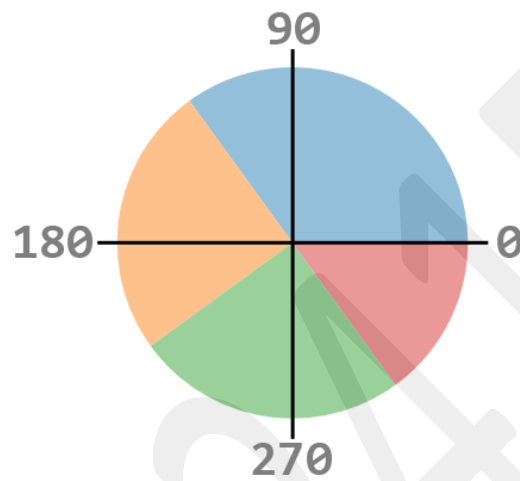
- The `labels` parameter assigns names to the slices, so each section is labeled.

3. Changing the Start Angle

By default, the first slice starts at **0 degrees** (aligned with the x-axis). You can change it with `startangle`.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]
# Start pie chart at 90 degrees
plt.pie(y, labels=labels, startangle=90)
plt.show()
```



Explanation:

- The `startangle=90` makes the first slice start from the **top (90 degrees)** instead of the default position.

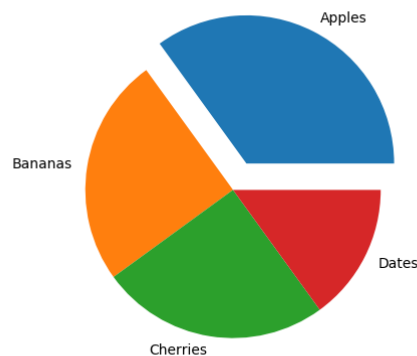
4. Highlighting a Slice (Exploding a Wedge)

You can "pull out" a slice to highlight it using `explode`.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]
explode = [0.2, 0, 0, 0] # Pull the first slice out

plt.pie(y, labels=labels, explode=explode)
plt.show()
```



Explanation:

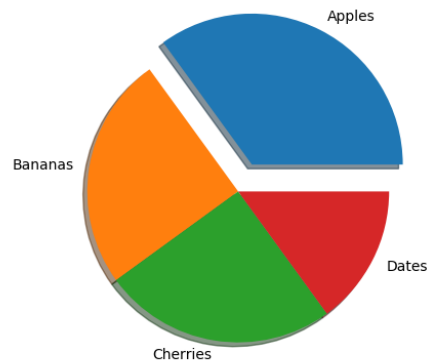
- The `explode` parameter moves the **first slice** out by **0.2 units**.
- The rest of the slices stay in place (0 values).

5. Adding a Shadow

Make the chart look 3D with a **shadow effect**.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]
explode = [0.2, 0, 0, 0]
plt.pie(y, labels=labels, explode=explode, shadow=True)
plt.show()
```



Explanation:

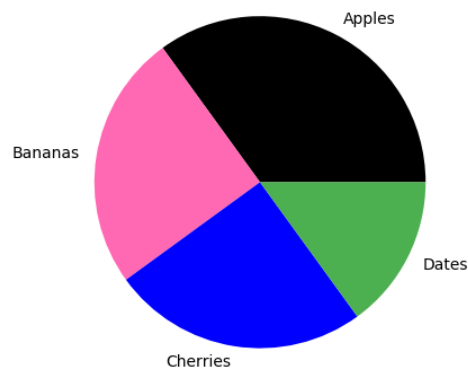
- `shadow=True` adds a shadow effect to make the chart look **more stylish**.

6. Customizing Colors

You can change slice colors using the `colors` parameter.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]
colors = ["black", "hotpink", "blue", "#4CAF50"] # Custom colors
plt.pie(y, labels=labels, colors=colors)
plt.show()
```



Explanation:

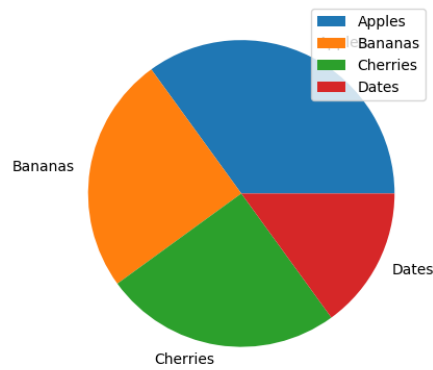
- You can use **color names**, **hex codes**, or **short color codes** like:
 - "r" for **red**
 - "g" for **green**
 - "b" for **blue**

7. Adding a Legend

A **legend** provides an explanation for each slice.

Example:

```
import matplotlib.pyplot as plt
import numpy as np
y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]
plt.pie(y, labels=labels)
plt.legend() # Add legend
plt.show()
```



Explanation:

- The `plt.legend()` function adds a **legend box** next to the chart.

8. Adding a Legend Title

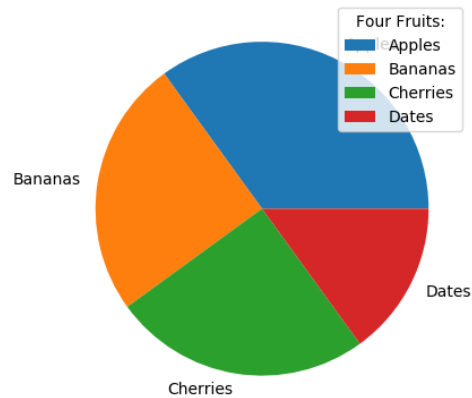
You can add a title to the legend using `title`.

Example:

```
import matplotlib.pyplot as plt
import numpy as np

y = np.array([35, 25, 25, 15])
labels = ["Apples", "Bananas", "Cherries", "Dates"]

plt.pie(y, labels=labels)
plt.legend(title="Fruits") # Add a title to the legend
plt.show()
```



Explanation:

- The **legend title** helps make the chart more descriptive.

Hira 2415